

O que é CSRF?

Cross-Site Request Forgery (CSRF, também chamado de XSRF ou *Sea Surf*) é um tipo de ataque no qual o atacante induz um usuário autenticado a executar, sem saber, uma ação em uma aplicação web na qual ele já está logado.

A premissa fundamental do ataque é simples: **o navegador envia os cookies do usuário automaticamente em toda requisição para o domínio correspondente**, independentemente de qual página disparou essa requisição. O servidor, ao receber a requisição com o cookie válido, não tem como distinguir se ela foi iniciada pelo próprio usuário ou por um script malicioso em outro site.

O CSRF foi catalogado pelo **OWASP** por muitos anos como um dos dez riscos mais críticos de segurança em aplicações web. A partir de 2021 saiu do Top 10 não porque deixou de ser perigoso, mas porque frameworks modernos passaram a incluir proteções por padrão — o que não significa que aplicações antigas ou mal configuradas estejam protegidas.

Como o ataque funciona — passo a passo

Contexto

- A vítima (**alice**) está autenticada no SafeBank (`localhost:3000`)
- O navegador de alice armazena um cookie `session_id=sess_xyz` para `localhost:3000`
- O atacante controla outro site (`localhost:4000`)

Fluxo detalhado

1. Alice acessa `localhost:3000` e faz login
 - Servidor cria sessão e responde com:
`Set-Cookie: session_id=sess_xyz; HttpOnly; Path=`
 - Navegador armazena o cookie
2. Alice, ainda logada, acessa `localhost:4000` (o site do atacante)
 - A página carrega normalmente (parece um site de prêmios)
 - JavaScript executa silenciosamente em segundo plano
3. O script submete um formulário HTML oculto:

```
<form action="http://localhost:3000/api/transfer" method="POST">
  <input name="to"      value="bob" />
  <input name="amount" value="500" />
```

```
</form>
```

→ O navegador envia a requisição para localhost:3000
→ INCLUI AUTOMATICAMENTE o cookie session_id=sess_xyz
(pois é uma requisição para localhost:3000, onde o cookie existe)

4. O servidor SafeBank recebe:

```
POST /api/transfer
```

```
Cookie: session_id=sess_xyz ← cookie legítimo de alice
```

```
Body: to=bob&amount=500
```

→ Valida o cookie → sessão existe → usuário é alice
→ Executa a transferência de R\$500 para bob
→ Retorna 200 OK

5. Alice nem percebe. A transferência foi realizada.

Por que o formulário HTML é o vetor ideal?

Formulários HTML (`<form method="POST">`) são submetidos pelo navegador como uma requisição HTTP simples. Eles **não estão sujeitos à política de CORS** (*Cross-Origin Resource Sharing*). Isso significa:

- O navegador envia a requisição sem fazer preflight
- Os cookies são incluídos normalmente (dependendo do SameSite)
- O atacante não consegue ler a resposta (bloqueado pelo CORS), mas isso não importa — a **ação já foi executada no servidor**

Essa é a diferença crucial: CORS protege a leitura de dados entre origens, mas não impede que uma ação seja disparada.

Vulnerabilidades presentes no SafeBank

Vulnerabilidade 1 — Ausência de token CSRF

O endpoint `POST /api/transfer` autentica o usuário exclusivamente pelo cookie de sessão:

```
// app/api/transfer/route.ts
const sessionId = request.cookies.get('session_id')?.value
if (!sessionId) return NextResponse.json({ error: 'Não autenticado' },
{ status: 401 })
```

```
const userId = sessions.get(sessionId)
// Se a sessão for válida, executa a transferência imediatamente
```

Não há nenhuma verificação de um token CSRF. Qualquer requisição com um cookie de sessão válido é aceita, independentemente de onde ela veio.

Por que isso é um problema? O cookie é enviado automaticamente pelo navegador — o atacante não precisa conhecê-lo nem roubá-lo. Basta que a vítima tenha o cookie e acesse a página maliciosa.

Vulnerabilidade 2 — Cookie sem SameSite=Strict

O cookie de sessão é criado sem o atributo SameSite:

```
// app/api/auth/login/route.ts
response.cookies.set('session_id', sessionId, {
  httpOnly: true,
  path: '/',
  // SameSite não definido
})
```

O atributo SameSite controla em quais contextos cross-site o navegador deve ou não enviar o cookie. Existem três valores:

Valor	Comportamento
Strict	Cookie nunca é enviado em requisições cross-site (nem navegação direta)
Lax	Cookie é enviado apenas em navegação top-level (clique em link) via GET. Bloqueia POST cross-site. Padrão moderno do Chrome
None	Cookie é sempre enviado, independentemente da origem. Requer Secure

Por que o ataque ainda funciona com o padrão Lax?

O atributo SameSite baseia-se no conceito de "**mesmo site**" (*same-site*), não de "**mesma origem**" (*same-origin*). A distinção é sutil:

- **Mesma origem:** mesmo esquema + mesmo host + mesma porta
(`http://localhost:3000` $\neq$ `http://localhost:4000`)
- **Mesmo site:** mesmo esquema + mesmo domínio registrável (eTLD+1)

Para `localhost`, o domínio registrável é simplesmente `localhost`. Portanto:

- `http://localhost:3000` e `http://localhost:4000` → **origens diferentes**, mas **mesmo site**

Com `SameSite=Lax` (padrão), o cookie **é enviado** em requisições entre portas diferentes de `localhost`, pois o navegador as considera do mesmo site. Logo, o formulário POST de `localhost:4000` para `localhost:3000` inclui o cookie normalmente.

Em um cenário real de produção, o atacante estaria em um domínio completamente diferente (ex: `premio-fake.com`), e `SameSite=Lax` bloquearia o POST. A correção correta é sempre usar `SameSite=Strict`.

Vulnerabilidade 3 — Endpoint não verifica o header `Origin`

Toda requisição cross-origin enviada por navegadores modernos inclui o header `Origin`, indicando de onde a requisição partiu. O servidor poderia usar isso para rejeitar requisições vindas de origens não autorizadas:

```
Origin: http://localhost:4000
```

O SafeBank ignora esse header completamente:

```
// Nenhuma verificação de Origin em nenhum endpoint
export async function POST(request: NextRequest) {
  const sessionId = request.cookies.get('session_id')?.value
  // Processa diretamente sem verificar de onde veio
```

Vulnerabilidade 4 — Endpoint aceita `application/x-www-form-urlencoded`

O endpoint aceita tanto JSON quanto dados de formulário:

```
const contentType = request.headers.get('content-type') ?? ''

if (contentType.includes('application/json')) {
  const body = await request.json()
  to = body.to
  amount = Number(body.amount)
} else {
  const formData = await request.formData() // ← aceita form submit
  to = formData.get('to') as string
  amount = Number(formData.get('amount'))
}
```

Isso é importante porque formulários HTML **só conseguem enviar** `application/x-www-form-urlencoded` ou `multipart/form-data` nativamente. Se o endpoint aceitasse apenas `application/json`, um form submit tradicional não funcionaria

(embora o atacante ainda pudesse usar `fetch` com `credentials: 'include'` em outros cenários).

Como corrigir cada vulnerabilidade

Correção 1 — Token CSRF (solução principal)

A proteção mais robusta contra CSRF é o padrão **Synchronizer Token Pattern**:

1. No login, o servidor gera um token CSRF aleatório e criptograficamente seguro, vinculado à sessão
2. Esse token é incluído em todo formulário como campo oculto (ou em requisições `fetch` via header)
3. O servidor valida que o token recebido corresponde ao da sessão antes de processar qualquer ação

```
// Geração do token no login
import { randomBytes } from 'crypto'

const csrfToken = randomBytes(32).toString('hex')
session.csrfToken = csrfToken

// Token enviado ao cliente (em cookie legível por JS ou no HTML)
response.cookies.set('csrf_token', csrfToken, {
  httpOnly: false, // precisa ser lido pelo JavaScript
  sameSite: 'strict',
  path: '/',
})

// Validação no endpoint sensível
export async function POST(request: NextRequest) {
  const sessionId = request.cookies.get('session_id')?.value
  const session = sessions.get(sessionId)

  // Token vindo do body ou do header X-CSRF-Token
  const csrfToken = request.headers.get('X-CSRF-Token')
    ?? (await request.json()).csrfToken

  if (!csrfToken || csrfToken !== session.csrfToken) {
    return NextResponse.json({ error: 'Token CSRF inválido' }, {
      status: 403 })
  }
  // ... prossegue com a transferência
}
```

Por que isso protege? O atacante não tem acesso ao token CSRF (política de mesma origem impede leitura de cookies ou do HTML do SafeBank a partir de outro site). Logo, não consegue incluí-lo na requisição forjada.

Correção 2 — Cookie com SameSite=Strict

```
response.cookies.set('session_id', sessionId, {
  httpOnly: true,
  path: '/',
  sameSite: 'strict', // ← bloqueia envio em qualquer contexto
  cross-site
  secure: true,        // ← requer HTTPS (obrigatório em produção)
})
```

Com SameSite=Strict, o cookie **não é enviado** em nenhuma requisição originada de outro site — nem mesmo em cliques diretos de links externos. Isso elimina completamente o vetor de CSRF baseado em cookies.

Cuidado: SameSite=Strict pode impactar a usabilidade. Se um usuário clicar em um link para o SafeBank vindo de um e-mail ou outro site, ele chegará deslogado (o cookie não é enviado na navegação inicial). Uma solução é usar SameSite=Lax para o cookie de sessão (aceitando a limitação de que protege apenas GET) combinado com token CSRF para ações sensíveis.

Correção 3 — Verificar o header Origin

```
export async function POST(request: NextRequest) {
  const origin = request.headers.get('origin')
  const allowedOrigin = 'https://safebank.com' // ou
  process.env.ALLOWED_ORIGIN

  if (origin && origin !== allowedOrigin) {
    return NextResponse.json({ error: 'Origem não autorizada' }, {
      status: 403 })
  }
  // ...
}
```

Essa verificação funciona como uma camada extra de defesa. É importante checar também o header `Referer` como fallback, pois alguns proxies corporativos removem o `Origin`.

Limitação: Requisições vindas diretamente de ferramentas como `curl` não enviam `Origin`. Por isso essa verificação não substitui o token CSRF — ela é uma camada adicional.

Correção 4 — Aceitar apenas `application/json`

Remover o suporte a `application/x-www-form-urlencoded` reduz a superfície de ataque, pois formulários HTML puros não conseguem enviar JSON com o `Content-Type` correto:

```
export async function POST(request: NextRequest) {
  const contentType = request.headers.get('content-type') ?? ''

  if (!contentType.includes('application/json')) {
    return NextResponse.json({ error: 'Content-Type inválido' }, {
      status: 415 })
  }

  const { to, amount } = await request.json()
  // ...
}
```

Atenção: Isso não é suficiente por si só. Um atacante pode usar `fetch` com `Content-Type: application/json` em navegadores que permitem `SameSite=None` ou se a `CORS` estiver mal configurada. Deve ser combinado com as outras correções.

Defesa em profundidade — aplicando todas as camadas

A proteção ideal combina múltiplas camadas independentes (*defense in depth*). Se uma falhar, as outras ainda protegem:

Camada 1: <code>SameSite=Strict</code> no cookie → Impede que o cookie seja enviado cross-site
Camada 2: Token CSRF validado no servidor → Mesmo que o cookie seja enviado, o token é desconhecido pelo atacante
Camada 3: Verificação do header <code>Origin/Referer</code> → Rejeita requisições de origens não autorizadas
Camada 4: Aceitar apenas <code>application/json</code> → Formulários HTML puros não conseguem forjar a req.

Diferença entre CSRF e XSS

Uma pergunta frequente é a diferença entre CSRF e XSS (*Cross-Site Scripting*):

	CSRF	XSS
O que explora	Confiança do servidor no navegador da vítima	Confiança do navegador da vítima no servidor
Precisa injetar código no alvo?	Não	Sim
Rouba credenciais/dados?	Não diretamente	Sim
Executa ações em nome da vítima?	Sim	Sim
Precisa que a vítima esteja logada?	Sim	Não necessariamente
Origem do script malicioso	Outro site	O próprio site alvo

No CSRF, o atacante **não lê dados** da aplicação — ele apenas **dispara ações**. Se a aplicação também tivesse XSS, o atacante poderia ler o token CSRF e contornar essa proteção, o que mostra por que as duas vulnerabilidades são complementares e ambas devem ser corrigidas.

Diferença entre CSRF e CORS

Outra confusão comum:

- **CORS** (*Cross-Origin Resource Sharing*) é um mecanismo do navegador que controla se scripts de uma origem podem **ler respostas** de outra origem. Ele **não impede** que a requisição seja enviada — apenas bloqueia a leitura da resposta pelo script.
- **CSRF** é um ataque que se aproveita exatamente disso: a requisição **é enviada e é processada** pelo servidor, mesmo que o script não consiga ler a resposta.

Por isso, configurar CORS restritivo **não protege contra CSRF**. Um formulário HTML não é afetado pelo CORS — ele envia a requisição normalmente e o atacante simplesmente não precisa ler a resposta para que o dano já esteja feito.

Resumo das vulnerabilidades e correções

Vulnerabilidade	Impacto	Correção
Cookie sem SameSite=Strict	Cookie enviado em requisições cross-site	sameSite: 'strict' no Set-Cookie
Ausência de token CSRF	Servidor aceita qualquer requisição com cookie válido	Gerar e validar token CSRF por sessão
Header Origin não verificado	Requisições de qualquer origem são aceitas	Rejeitar Origin fora da allowlist
Aceita form-urlencoded	Formulários HTML podem disparar a ação	Aceitar apenas application/json